Bachelor of Computer Applications (Hons. AI and ML)
Semester- III

Subject: Database Management System

Assignment 4: Relational Database vs. NoSQL Database

By
Name: AKASH P
Reg. No.: 2511021240008

Faculty Name: Renita Joy
Faculty Signature:_____

Department of Computer Application
Alliance University
Chandapura - Anekal Main Road, Anekal
Bengaluru - 562 106
October 2026

1. Introduction

Relational and NoSQL databases solve overlapping but different data-management problems. The best choice depends on data shape, consistency needs, query patterns, operational skills, and growth requirements. The Online Food Delivery System in this project is naturally relational because it contains structured entities (customers, restaurants, menu items, orders, payments, delivery agents, and reviews) with clear foreign-key relationships.

2. Comparison

Feature	Relational Database (RDBMS)	NoSQL Database
Scalability	Vertical scaling (bigger servers). Can scale horizontally, but this is more complex.	Built for horizontal scaling. Easily handles large volumes of data across multiple servers.
Consistency	Strong consistency (ACID properties). Ensures data accuracy and integrity.	Eventual consistency in most cases. May have slight delays in data propagation, depending on the database type.
Performance	Very good for complex queries. Performance can decrease with very large data volumes or high traffic.	High performance for read/write operations. Optimized for specific use cases (document, key-value, column, graph).
Data Model	Structured (tables, rows, columns) with a fixed schema.	Flexible (document, key-value, column, graph). Schema-less or dynamic schema.
Complex Queries	Excellent support for complex joins, aggregations and transactions.	Limited or no support for joins. Simpler queries, optimized for specific patterns.
Best Suited For	Financial systems, e-commerce, ERP, CRM, transaction-heavy applications.	Big data, real-time apps, social media, content management, IoT, gaming.

Real-world examples of each type:

Database	Type	Commonly used for
MySQL	Relational	Web applications, e-commerce
PostgreSQL	Relational	Enterprise systems, analytics
Oracle	Relational	Large enterprises, transaction systems
SQL Server	Relational	Business applications, Windows ecosystem
MongoDB	NoSQL (document)	Content management, e-commerce
Cassandra	NoSQL (wide-column)	Big data, logging, real-time analytics
Redis	NoSQL (key-value)	Caching, real-time applications
Neo4j	NoSQL (graph)	Graph data, social networks

3. Real-world examples

Relational examples: MySQL and PostgreSQL are commonly used for business systems with structured records and transactions. A food delivery system can use foreign keys to ensure that an order references an existing customer, restaurant, and delivery address, and that a payment or review belongs to a real order. Financial and order-processing systems also commonly use relational transactions to keep related updates consistent, such as saving an order, its items, and its payment together or not at all.

NoSQL examples: MongoDB is a document database suited to flexible document-shaped records; Redis is a key-value store commonly used for caching and fast ephemeral lookups; Apache Cassandra is a wide-column database designed for distributed, high-write workloads; Neo4j is a graph database suited to highly connected data such as recommendation or network relationships. These examples describe common fit, not a claim that each organization uses only that technology.

4. Why organizations choose different technologies

Organizations choose based on requirements rather than on a database being universally superior. A relational database is a good default when relationships, constraints, and multi-table transactions are central. A NoSQL database may be appropriate when the workload benefits from flexible records, specific access patterns, large distributed write volume, or graph traversal. Some systems use multiple databases for different workloads, a pattern called polyglot persistence.

5. Recommendation for the proposed Online Food Delivery System

Use a relational database such as MySQL or PostgreSQL for the current project. Customers, Addresses, Restaurants, Menu_Items, Orders, Order_Items, Payments, and Delivery_Agents have clear relationships and require referential integrity. Placing an order touches several tables at once (order, order lines, payment), so ACID transactions are important to keep money and order status correct. SQL joins also make the required reports straightforward, such as orders per restaurant, delivery agent workload, payment status, and average review rating.

A NoSQL store is not necessary for the core system based on the scope provided. It could be considered later for separate features with different needs, for example Redis to cache popular restaurant menus, a high-write store for live delivery-agent location updates, or a graph database for restaurant recommendations. The order, payment and delivery records themselves should remain in the relational database.

6. Conclusion

Relational databases prioritize structured relationships and transactional integrity; NoSQL databases offer several alternative models that can suit particular scaling and data-shape needs. The database should be selected to fit the application’s actual workload, consistency requirements, and team capabilities. For this food delivery system, a relational design is the better fit for its core data.